

Báo cáo Nghiên cứu Chuyên sâu: Thiết kế Vi kiến trúc và Tối ưu hóa Hệ thống Serving vLLM cho LFM2.5-1.2B dưới Giới hạn CPU Cực đoan

1. Phân tích Hiện trạng và Nhận diện Điểm nghẽn Cấu trúc (Structural Bottleneck Analysis)

Việc tối ưu hóa một hệ thống phục vụ mô hình ngôn ngữ lớn (LLM inference server) đòi hỏi một sự hiểu biết sâu sắc không chỉ về phần mềm mà còn về sự tương tác vật lý giữa các thành phần phần cứng. Trong bối cảnh bài toán triển khai mô hình LiquidAI/LFM2.5-1.2B-Instruct trên một phân vùng MiG (Multi-Instance GPU) của kiến trúc NVIDIA H200, sự bất đối xứng về tài nguyên tạo ra một môi trường tối ưu hóa cực kỳ đặc thù. Việc nhận diện sai điểm nghẽn (bottleneck) sẽ dẫn đến việc áp dụng các kỹ thuật phân tác dụng, làm suy giảm trực tiếp Điểm Yêu cầu Hiệu quả (Effective Request Score - ERS).

1.1. Nghịch lý Bất đối xứng Phần cứng (The Hardware Asymmetry Paradox)

Hệ thống được cấp phát sở hữu một lát cắt MiG H200 cung cấp 18GB VRAM.¹ Kiến trúc H200 (Hopper) sở hữu các lõi Tensor Core thế hệ thứ 4 với sức mạnh tính toán khổng lồ và băng thông bộ nhớ bộ nhớ băng thông cao (HBM3e) vượt trội.² Tuy nhiên, năng lực điện toán khổng lồ này lại bị trói buộc bởi một hệ thống máy chủ (host) bị giới hạn nghiêm ngặt: chỉ có 3 lõi CPU (Cores) và 8GB RAM bộ nhớ hệ thống (System RAM).

Mô hình LFM2.5-1.2B-Instruct có kích thước xấp xỉ 1.17 tỷ tham số.³ Nếu hệ thống tải toàn bộ mô hình ở định dạng độ chính xác BF16 (Bfloat16), không gian VRAM bị chiếm dụng chỉ vào khoảng 2.34 GB. Ngay cả khi phục vụ 70 hội thoại đồng thời (kịch bản tải cao nhất theo phân phối Poisson) với 4000 token ngữ cảnh (context), dung lượng bộ nhớ (KV Cache) ước tính cũng chỉ tiêu tốn thêm khoảng 1.5 - 2.5 GB do đặc thù kiến trúc lai của mô hình.¹ Tổng dung lượng VRAM thực tế cần thiết để hệ thống vận hành trơn tru hiếm khi vượt qua mức 5 GB đến 6 GB, để lại hơn 12 GB VRAM trên phân vùng MiG hoàn toàn nhàn rỗi.

Nghịch lý nảy sinh từ đây: tài nguyên GPU đang ở trạng thái dư thừa (Over-provisioned), trong khi tài nguyên CPU lại ở trạng thái cạn kiệt (Starved). Trong các khung phục vụ (serving frameworks) phổ biến như vLLM, CPU đóng vai trò như một nhạc trưởng điều phối toàn bộ hệ thống.⁵ CPU phải xử lý các kết nối mạng HTTP/REST, phân tích chuỗi văn bản đầu vào thành token (Tokenization), quản lý vòng đời của các yêu cầu, cấp phát các khối bộ nhớ PagedAttention, và quan trọng nhất là liên tục phóng (launch) hàng ngàn hạt nhân CUDA (CUDA kernels) lên GPU cho mỗi bước giải mã (decode step).⁵ Khi GPU H200 hoàn thành công

việc tính toán một token đầu ra nhanh hơn tốc độ mà 3 lõi CPU có thể chuẩn bị và gửi lệnh cho token tiếp theo, GPU sẽ rơi vào trạng thái "đói dữ liệu" (idle) và buộc phải chờ đợi.⁶ Hiện tượng này được gọi là Độ trễ Lập lịch do thắt cổ chai CPU (CPU-Bound Scheduling Overhead), khiến chỉ số Thời gian sinh mỗi Token đầu ra (TPOT) tăng vọt một cách giả tạo, bất chấp sức mạnh của phần cứng đồ họa.

1.2. Giải phẫu Kiến trúc LFM2.5 và Thiết kế Bộ đệm Trạng thái

Để khai thác tối đa tiềm năng của vLLM và giải phóng CPU, việc hiểu rõ kiến trúc của mô hình LFM2.5 (Liquid Foundation Models) là bắt buộc. Khác với các mô hình phổ thông như Llama hay Qwen vốn sử dụng kiến trúc Transformer thuần túy, LFM2.5 là một hệ thống lai (Hybrid Architecture) được thiết kế đặc biệt cho việc triển khai trên các thiết bị có tài nguyên hạn chế (Edge Devices).³

Mô hình LFM2.5-1.2B bao gồm tổng cộng 16 lớp xử lý.³ Cấu trúc này được chia thành hai phần riêng biệt:

1. 10 lớp Tích chập cổng kép (Double-gated LIV convolution blocks): Các lớp này xử lý luồng dữ liệu tuần tự và lưu trữ thông tin dưới dạng trạng thái chập (convolutional state), tương tự như kiến trúc Mamba, không sử dụng cơ chế Attention truyền thống.¹⁰
2. 6 lớp GQA (Grouped Query Attention blocks): Chỉ có 6 lớp này tham gia vào việc tạo ra Key-Value (KV) Cache truyền thống theo cơ chế của PagedAttention.¹

Sự phân bố kiến trúc này mang lại một lợi thế khổng lồ về mặt quản lý bộ nhớ: Dung lượng KV Cache được tạo ra trên mỗi token chỉ bằng khoảng 37.5% (6/16) so với một mô hình Transformer thuần túy có cùng số lớp và cùng $\bar{x}$ nguyên (hidden size). vLLM quản lý trạng thái chập (convolutional state) thông qua các lớp được tùy chỉnh đặc biệt như Lfm2ForCausalLM và tính toán hình dáng cache qua module MambaStateShapeCalculator.¹⁰ Nhờ KV Cache quá nhỏ, cơ chế Prefix Caching (vốn cực kỳ hữu ích cho 70 hội thoại có chung 15 primer hoặc chung system prompt) có thể lưu giữ toàn bộ cấu trúc dữ liệu trên VRAM mà không bao giờ cần phải hoán đổi (offload/swap) xuống CPU hoặc RAM hệ thống.¹³ Hệ thống RAM 8GB hiện tại không có không gian cho việc offload (swap), do đó sự nhỏ gọn của LFM2.5-1.2B chính là chìa khóa để giữ mọi dữ liệu tính toán hoàn toàn nằm trên GPU.

1.3. Cơ sở Toán học của Hệ thống Chấm điểm ERS

Chiến lược tối ưu hóa phải được điều hướng chặt chẽ bởi công thức chấm điểm ERS (Effective Request Score). Hệ thống ERS tập trung hoàn toàn vào độ trễ (TTFT chiếm 50% trọng số, TPOT chiếm 50% trọng số) dựa trên hàm suy hao đa thức:

$$S_{request} = w \cdot (x_{ttft})^\gamma + (1 - w) \cdot (x_{tpot})^\gamma$$

Với các tham số: $w = 0.5$, $\gamma = 2$, Floor TTFT (F_{ttft}) = 10ms, Ceiling TTFT (C_{ttft}) = 400ms, Floor TPOT (F_{tpot}) = 1ms, Ceiling TPOT (C_{tpot}) = 10ms.

Hàm x_{ttft} và x_{tpot} sử dụng hàm kẹp (clamp) để giới hạn giá trị trong khoảng \$\$\$. Đặc biệt,

việc sử dụng hệ số lũy thừa $\gamma = 2$ có nghĩa là điểm số không suy giảm tuyến tính mà sẽ bị trừng phạt theo đường cong parabol (quadratic penalty). Nếu TTFT hoặc TPOT vượt qua giá trị Floor chỉ một lượng nhỏ, điểm số sẽ tụt dốc không phanh. Một TPOT đạt $5.5ms$ (nằm giữa Floor và Ceiling) sẽ không mang lại 0.5 điểm cho thành phần đó, mà chỉ mang lại $(0.5)^2 = 0.25$ điểm. Do đó, mục tiêu tối thượng không phải là đạt tốc độ "đủ nhanh", mà là đẩy TTFT tiệm cận giới hạn vật lý $10ms$ và TPOT tiệm cận $1ms$. Mọi nỗ lực tối ưu hóa phải được thiết kế để cắt giảm độ trễ ở mức micro-giây (microseconds) bằng cách gỡ bỏ mọi rào cản giao tiếp giữa các tiến trình.

2. Giải tỏa Điểm nghẽn Cổ chai CPU: Tái cấu trúc Execution Pipeline

Việc cấu hình vLLM theo các thiết lập mặc định hoặc bật hàng loạt các cờ (flags) tối ưu hóa một cách mù quáng chính là nguyên nhân dẫn đến sự suy giảm hiệu năng. Nhiều kỹ thuật tối ưu hóa được sinh ra để tiết kiệm VRAM, nhưng trong bối cảnh này, chúng lại tiêu tốn CPU. Các bước dưới đây sẽ thay đổi triệt để cách GPU và CPU tương tác.

2.1. Nghịch lý của Chunked Prefill và Sự phục sinh của CUDA Graphs

Hệ thống hiện tại đang được cấu hình sử dụng tính năng Chunked Prefill. Mặc dù Chunked Prefill là một kỹ thuật xuất sắc để kiểm soát độ trễ TTFT bằng cách chia nhỏ quá trình mỗi ngữ cảnh (prefill) của các yêu cầu dài thành nhiều khối (chunks) và đan xen chúng với các yêu cầu đang trong giai đoạn giải mã (decode)¹⁴, nó lại là một "chất độc" đối với kiến trúc 3 lõi CPU. Khi Chunked Prefill được kích hoạt, hệ thống lập lịch của vLLM sẽ liên tục thay đổi kích thước của lô (batch size) và chiều dài chuỗi (sequence length) theo từng bước chạy. Các hình dạng tensor linh động (dynamic tensor shapes) này phá vỡ hoàn toàn khả năng sử dụng **CUDA Graphs**.⁷

CUDA Graphs là một cơ chế tối ưu hóa then chốt của PyTorch và NVIDIA. Thay vì CPU phải gửi từng lệnh gọi hạt nhân CUDA (kernel launch) riêng lẻ cho từng thao tác toán học trong mạng nơ-ron (nhân ma trận, chuẩn hóa, kích hoạt), CUDA Graphs cho phép CPU ghi lại (capture) toàn bộ chuỗi hàng trăm lệnh gọi này vào một đồ thị duy nhất. Trong các bước giải mã tiếp theo, CPU chỉ cần phát ra đúng một lệnh duy nhất để kích hoạt toàn bộ đồ thị trên GPU.⁷ Việc này loại bỏ hoàn toàn chi phí (overhead) giao tiếp giữa CPU và GPU, giảm thời gian chết của GPU từ vài phần nghìn giây xuống còn vài micro-giây. Tuy nhiên, giới hạn cốt lõi của CUDA Graphs là nó yêu cầu hình dạng tensor phải hoàn toàn tĩnh (static shapes). Nếu kích thước batch thay đổi liên tục do Chunked Prefill, vLLM không thể đối sánh (match) biểu đồ đã lưu và sẽ rơi về chế độ thực thi PyTorch Eager Mode.¹³

Ở chế độ Eager Mode, đối với mỗi lớp trong số 16 lớp của LFM2.5, CPU phải gửi đi một lệnh gọi hạt nhân riêng biệt. Với 3 lõi CPU phải phục vụ 70 requests đến theo phân phối Poisson, hàng đợi của CPU (CPU queue) sẽ ngay lập tức bị quá tải, dẫn đến GPU luôn phải chờ đợi và khiến TPOT tăng vọt không kiểm soát.

Chiến lược Can thiệp Cốt lõi:

- 1. Tắt hoàn toàn Chunked Prefill (--disable-chunked-prefill hoặc không truyền cờ kích hoạt).
- 2. Kích hoạt explicitly cơ chế ép buộc đồ thị bằng cờ --enforce-eager=False. Tham số này đảm bảo rằng vLLM sẽ nỗ lực tối đa để ghi lại và biên dịch (torch.compile) các biểu đồ CUDA cho các kích thước batch khác nhau lúc khởi động (warm-up).¹³
- 3. Để đối phó với luồng 70 request đến đồng loạt, thiết lập --max-num-batched-tokens trùng hoặc vượt quá số lượng token tối đa có thể phát sinh. Với VRAM dư thừa, hệ thống hoàn toàn có thể hấp thụ toàn bộ sự bùng nổ (burst) của luồng Poisson trong một chu kỳ Prefill duy nhất mà không cần phải chia nhỏ (chunk) chúng ra.

Kỹ thuật Thiết lập	Trạng thái CUDA Graphs	Chi phí CPU (CPU Overhead)	Tác động lên TPOT
Sử dụng Chunked Prefill	Vô hiệu hóa (Eager Mode)	Rất Cao (Liên tục gửi kernel)	TPOT tăng vọt do GPU đổi dữ liệu
Tắt Chunked Prefill	Hoạt động (Graph Replay)	Rất Thấp (1 lệnh Launch)	TPOT giảm kịch trần (Tiệm cận 1ms)

2.2. Kích hoạt Khung Thực thi Model Runner V2 (MRV2)

vLLM gần đây đã giới thiệu một kiến trúc thực thi nội bộ hoàn toàn mới có tên là MRV2 (Model Runner V2).²⁰ Thiết kế của phiên bản V1 trước đây phụ thuộc rất nhiều vào CPU để thực hiện các công việc chuẩn bị siêu dữ liệu đầu vào (input metadata preparation), lấy mẫu (sampling) và sổ sách quản lý lô liên tục (persistent batch bookkeeping).²¹ Việc chuẩn bị các mảng như input_ids, độ dài chuỗi (seq_lens), và con trỏ khối (block pointers) yêu cầu hàng ngàn phép toán nhỏ nhặt trên CPU.

Nguyên lý thiết kế của MRV2 là "Be GPU-native" (Bản địa hóa trên GPU). MRV2 sử dụng các hạt nhân Triton tùy chỉnh để chuẩn bị dữ liệu cấu trúc đầu vào và thực hiện lấy mẫu xác suất trực tiếp trên VRAM của GPU, bỏ qua hoàn toàn việc trao đổi mảng dữ liệu giữa RAM hệ thống và VRAM thông qua bus PCIe.²⁰ Hơn nữa, MRV2 sử dụng CUDAGraphManager để kiểm soát vòng đời của đồ thị CUDA một cách rõ ràng, mang lại sự ổn định và hiệu năng cao hơn nhiều so với việc chắp vá V1.

Hành động: Thiết lập biến môi trường export VLLM_USE_V2_MODEL_RUNNER=1 trước khi khởi chạy tiến trình. Điều này sẽ ngay lập tức "bốc hơi" một lượng lớn chu kỳ tính toán khỏi 3 lõi CPU nghèo nàn, biến CPU trở thành một người giao liên thuần túy thay vì một người thợ sắp xếp dữ liệu.

2.3. Tối thiểu hóa Đồng bộ hóa với Multi-Step Scheduling (Dual Batch Overlap)

Ngay cả khi CUDA Graphs và MRV2 được kích hoạt, theo cơ chế mặc định của LLM inference,

sau khi sinh ra một token (kết thúc một decode step), GPU vẫn phải báo cáo lại kết quả cho CPU. CPU sau đó cập nhật bộ đệm trạng thái, kiểm tra điều kiện dừng (stop criteria), và kích hoạt lệnh giải mã cho token tiếp theo. Quá trình Ping-Pong đồng bộ hóa (CPU-GPU synchronization) này mất một khoảng thời gian nhất định (thường là vài chục đến hàng trăm micro-giây).⁷ Nếu mục tiêu TPOT là $1ms$, việc mất $0.2ms$ cho việc đồng bộ hóa đã tiêu tốn **20%** ngân sách thời gian quý giá.

Công nghệ "Dual Batch Overlap" (DBO) hay còn gọi là Lập lịch Đa bước (Multi-Step Scheduling) giải quyết triệt để rào cản này. Bằng cách sử dụng tham số `--num-scheduler-steps`, hệ thống cho phép cỗ máy nội bộ trên GPU tự động thực thi một chuỗi nhiều bước giải mã liên tiếp (ví dụ: 10 bước) trước khi cần phải báo cáo trạng thái trở lại cho tiến trình trên CPU.¹³ Trong suốt quá trình này, GPU tự động cập nhật các bộ đếm và con trỏ bằng các hạt nhân nội bộ (in-place GPU updates).

- Việc thiết lập `--num-scheduler-steps 10` sẽ gộp 10 chu kỳ decode vào một lệnh launch duy nhất từ CPU. Nghĩa là số lần CPU phải đồng bộ hóa với GPU giảm đi 90%.
- Đánh đổi duy nhất là hệ thống sẽ mất khả năng phản hồi luồng (streaming) ngay lập tức theo từng token đơn lẻ. Tuy nhiên, hệ thống đo lường ERS không quan tâm đến tính mượt mà của giao diện người dùng mà chỉ quan tâm đến tốc độ hoàn thành. Do đó, việc đẩy số lượng bước lập lịch lên cao là chìa khóa vàng để TPOT phá vỡ giới hạn.

3. Quản lý Mạng, Trình diễn (Frontend) và Giao tiếp Liên tiến trình (IPC)

Việc nhận được 70 request đến dồn dập (Poisson arrival) yêu cầu một tầng mạng (network layer) xử lý bất đồng bộ cực kỳ xuất sắc. Hệ thống đang áp dụng `VLLM_USE_RUST_FRONTEND=1` kết hợp với `TOKIO_WORKER_THREADS=2`. Đây là một bước đi hoàn toàn chính xác để vượt qua giới hạn của Python GIL (Global Interpreter Lock) và cấu trúc FastAPI/Uvicorn truyền thống.²³ Môi trường Rust với trình lập lịch Tokio cung cấp khả năng xử lý concurrency tuyệt vời.²⁶ Tuy nhiên, vẫn còn dư địa kiến trúc để tối đa hóa băng thông giữa Frontend (Rust) và Backend (Python Engine).

3.1. Ứng dụng Unix Domain Sockets (UDS) thay thế TCP Loopback

Theo mặc định, luồng Rust frontend và tiến trình Python vLLM Engine giao tiếp với nhau thông qua mạng TCP/IP loopback (cụ thể là liên kết qua địa chỉ `http://localhost:8000` hoặc tương tự). Quá trình truyền dữ liệu qua TCP/IP đòi hỏi dữ liệu phải di chuyển qua toàn bộ ngăn xếp mạng (network stack) của hệ điều hành Linux.²⁷ Các bước này bao gồm đóng gói (encapsulation), tính toán mã kiểm tra (checksum), định tuyến nội bộ, và sao chép dữ liệu giữa không gian người dùng (user space) và không gian nhân (kernel space). Đối với một máy chủ chỉ có 3 lõi CPU, việc xử lý hàng chục ngàn gói tin TCP mỗi giây cho 70 hội thoại tạo ra một overhead khổng lồ. Unix Domain Sockets (UDS) là giao thức IPC được thiết kế để bỏ qua toàn bộ ngăn xếp mạng mạng phức tạp này.²⁸ UDS cho phép hai tiến trình trên cùng một máy chủ trao đổi dữ liệu trực tiếp thông qua một tệp socket (socket file) được quản lý trong bộ nhớ kernel, loại bỏ việc tính

toán checksum và giảm thiểu số lần chuyển đổi ngữ cảnh (context switches).²⁷

Chiến lược Triển khai: Sử dụng cờ `--uds /tmp/vllm.sock` khi khởi chạy vLLM.²⁸ Tham số này sẽ ra lệnh cho hệ thống bỏ qua việc ràng buộc (bind) các cổng TCP (host/port) và sử dụng đường ống trực tiếp. Điều này giúp tiến trình Rust frontend đẩy luồng văn bản và cấu hình trực tiếp vào engine xử lý với độ trễ gần như bằng không.

3.2. Tích hợp Fasttokens và Giảm tải Xử lý Chuỗi

Quá trình phân tích từ vựng (Tokenization) là một tác vụ nặng về thao tác chuỗi (string manipulation) và được thực thi hoàn toàn trên CPU. Mặc dù cấu hình hiện tại đã sử dụng tokenizer dựa trên nền tảng Rust của thư viện HuggingFace (tokenizers), luồng dữ liệu vẫn phải băng qua cầu nối Python (Python bindings) trước khi được đưa vào engine.³⁴

Việc sử dụng cờ môi trường `VLLM_USE_FASTTOKENS=1` tích hợp thư viện Fasttokens, cho phép hoán đổi trực tiếp (monkey-patch) bộ giải mã bên trong.³⁴ Cấu trúc này cho phép luồng Rust frontend và C++ engine thực hiện mã hóa và giải mã chuỗi mà không bao giờ cần đánh thức (wake up) trình thông dịch Python. Để tối ưu hóa, hãy kết hợp với cờ `--tokenizer-mode hf` để đảm bảo hệ thống tải đúng bộ mã hóa tốc độ cao của HuggingFace, tạo thành một chuỗi cung ứng I/O hoàn hảo dưới sự điều phối của 2 luồng Tokio (`TOKIO_WORKER_THREADS=2`).²⁶ Việc giữ `OMP_NUM_THREADS=1` là hoàn toàn đúng đắn, tránh hiện tượng cạnh tranh luồng (thread thrashing) giữa OpenMP và Tokio trên một CPU chỉ có 3 lõi.

3.3. Tối ưu hóa Trình Cấp phát Bộ nhớ với TCMalloc

Với chỉ 8GB RAM bộ nhớ hệ thống, vLLM phải rất chặt vật để duy trì không gian hoạt động cho cả hệ điều hành, trình thông dịch Python, Runtime của Rust, và các thư viện CUDA. Trình cấp phát bộ nhớ mặc định của hệ điều hành Ubuntu (glibc malloc) thường gặp vấn đề phân mảnh bộ nhớ (Memory Fragmentation) và tranh chấp khóa (Lock Contention) khi chạy các ứng dụng đa luồng mật độ cao như LLM inference.³⁹

Khi malloc không tìm được khối nhớ liền kề, nó phải yêu cầu hệ điều hành phân trang (page fault), khiến CPU bị đình trệ. TCMalloc (Thread-Caching Malloc) của Google là một giải pháp cấp phát bộ nhớ siêu tốc giải quyết triệt để vấn đề này. Nhờ cấu trúc lưu trữ cục bộ cho từng luồng (thread-local cache), TCMalloc cho phép cấp phát và giải phóng các đối tượng bộ nhớ nhỏ (như token IDs, request objects) mà không cần khóa toàn cục (global locks).³⁹

Chiến lược Triển khai: Cài đặt thư viện `libtcmalloc-minimal4` trong tệp Dockerfile và thiết lập biến môi trường liên kết động trước khi khởi chạy vLLM: `export`

`LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libtcmalloc_minimal.so.4:$LD_PRELOAD`.⁴⁰ Sự tinh chỉnh OS-level này sẽ kéo giảm trực tiếp phần trăm CPU bị chiếm dụng bởi nhân hệ điều hành (sys time), nhường chu kỳ xử lý cho việc điều phối GPU.

4. Chiến lược Lượng tử hóa Nghịch đảo (Inverse Quantization Strategy)

Bài toán yêu cầu hy sinh hoàn toàn độ chính xác (Accuracy) để đổi lấy tốc độ trong vòng đo lường Online. Hệ thống hiện tại đang sử dụng lượng tử hóa FP8 cho cả mô hình (model weights) và bộ đệm (KV cache fp8). Cần phân tích sâu sự khác biệt giữa hai quyết định này dựa trên nền

tảng phần cứng H200.

4.1. Sự Hoàn hảo của Trọng số FP8 (FP8 Model Weights)

NVIDIA H200 (kiến trúc Hopper) là con quái vật phần cứng được sinh ra để tính toán các ma trận định dạng FP8 (E4M3/E5M2).⁴⁴ Trọng số của mô hình khi được lượng tử hóa xuống FP8 mang lại hai lợi ích to lớn:

- 1. Giảm một nửa lượng dữ liệu trọng số (weights) phải được tải từ bộ nhớ HBM vào SRAM của GPU cho mỗi thao tác nhân ma trận (Matrix Multiplication). Việc giải quyết thắt cổ chai băng thông bộ nhớ (Memory Bandwidth Bound) này tăng mạnh Tốc độ Sinh Token (Token Generation Rate).
- 2. Kích hoạt các hạt nhân phần cứng chuyên biệt (Tensor Core FP8 W8A8) thực hiện phép toán cực nhanh. So với định dạng INT8 SmoothQuant cũ kỹ, FP8 tận dụng phần cứng native tốt hơn hẳn.⁴⁶ Vì vậy, việc sử dụng trọng số mô hình FP8 là một chiến lược bắt buộc và hoàn toàn chính xác.

4.2. Sai lầm Chết người của KV Cache FP8 trong Ngữ cảnh Dư thừa VRAM

Ngược lại với trọng số, việc thiết lập KV Cache thành định dạng FP8 (--kv-cache-dtype fp8) lại là một sai lầm cấu trúc nghiêm trọng trong bài toán này. Kỹ thuật lượng tử hóa KV Cache sinh ra với một mục đích duy nhất: **Tiết kiệm VRAM** để cho phép tăng số lượng request xử lý đồng thời (Batch Size) khi hệ thống bị tràn bộ nhớ.¹³

Tuy nhiên, như đã phân tích ở Phần 1, hệ thống MiG H200 hiện tại đang có 18GB VRAM, trong khi mô hình LFM2.5-1.2B chỉ chiếm khoảng 1.2GB trọng số (sau khi lượng tử hóa FP8) và khoảng 2GB KV Cache (do kiến trúc GQA với chỉ 6 lớp attention).¹ Hệ thống vẫn còn trống hơn 14GB VRAM!

Khi kích hoạt KV Cache FP8, để thực hiện phép toán tính toán Attention, GPU không thể tính toán trực tiếp mà không trải qua quá trình chuyển đổi. vLLM phải sử dụng các nhân CUDA đặc biệt để liên tục ép kiểu (casting) các giá trị kích hoạt (activations) và tính toán lại các hệ số tỷ lệ (scaling factors) trên đường bay (on-the-fly).⁴⁸ Quá trình này tạo ra một "Compute Overhead" (Độ trễ do tính toán dư thừa) rất lớn trên GPU. Mặc dù GPU rất mạnh, nhưng việc bắt nó thực hiện các phép toán vô nghĩa để tiết kiệm dung lượng VRAM (vốn không hề thiếu) sẽ trực tiếp kéo lùi TPOT.⁴⁸

Chiến lược Can thiệp:

Giữ nguyên trọng số ở định dạng lượng tử FP8 để tăng tốc độ nhân ma trận, nhưng cấu hình **KV Cache giữ nguyên ở định dạng BF16** (--kv-cache-dtype auto hoặc bf16). Việc giữ KV Cache ở định dạng nguyên bản giúp hệ thống bỏ qua hoàn toàn chi phí ép kiểu và tính toán hệ số trong bước Decode. Sự đánh đổi này (sử dụng thêm khoảng 2GB VRAM trống để đổi lấy hàng trăm micro-giây TPOT) là công thức hoàn hảo để tối đa hóa ERS.

Cấu hình KV Cache	VRAM Tiêu thụ	Overhead Tính toán (GPU)	TPOT (Thời gian sinh Token)	Khuyến nghị

		Compute)		
FP8 (W8A8)	Rất thấp (~1 GB)	Cao (Ép kiểu và tính Scaling Factor liên tục)	Cao (Bị kim hãm)	KHÔNG SỬ DỤNG
BF16 / Auto	Thấp (~2.5 GB)	Gần như Không có (Native Format)	Rất Thấp (Nhanh nhất)	BẮT BUỘC SỬ DỤNG

5. Tối ưu hóa Attention Backend: Sức mạnh của FlashInfer

Thư viện xử lý Attention (Cơ chế chú ý) đóng vai trò định đoạt tốc độ của khâu Prefill (TTFT) và Decode (TPOT). Trên các dòng GPU thế hệ mới như H100 và H200 (kiến trúc Hopper), vLLM thường ưu tiên sử dụng FlashAttention-3 hoặc XFormers cho các luồng xử lý thông thường.² Mặc dù FlashAttention-3 cực kỳ mạnh mẽ trong việc xử lý Attention truyền thống (Multi-Head Attention), thiết kế của nó không hoàn toàn tối ưu hóa cho cấu trúc PagedAttention bị phân mảnh (Block-sparse KV cache) và cơ chế GQA (Grouped-Query Attention).²

Như đã giải phẫu ở Phần 1, LFM2.5-1.2B sử dụng 6 lớp GQA để tính toán Attention.³ GQA có đặc điểm là nhiều Head của Truy vấn (Query) sẽ chia sẻ chung một Head của Khóa/Giá trị (Key/Value). Nếu backend không được thiết kế chuyên biệt, nó sẽ phải thực hiện các phép tính nhân rộng (broadcasting) tốn kém.

Thư viện FlashInfer được phát triển với mục tiêu chuyên biệt hóa cho việc phục vụ LLM (Inference Serving). Điểm mạnh cốt lõi của FlashInfer là nó sở hữu các hạt nhân (kernels) được tùy biến sâu cho môi trường Grouped-Query Attention, Multi-head latent attention (MLA), và xử lý mượt mà các khối KV Cache không liên tục (PagedAttention).² Các nghiên cứu độc lập chỉ ra rằng trên nền tảng Hopper, FlashInfer có thể mang lại thông lượng Decode (TPOT) cao hơn FlashAttention truyền thống nhờ giảm thiểu độ trễ phóng hạt nhân (kernel launch overhead) và tối ưu hóa truy cập bộ nhớ dùng chung (SRAM).²

Chiến lược Triển khai: Sử dụng biến môi trường để buộc vLLM bỏ qua backend mặc định và định tuyến (route) toàn bộ quá trình Attention qua thư viện FlashInfer: `export VLLM_ATTENTION_BACKEND=FLASHINFER`.⁵² *Lưu ý Quan trọng:* Quyết định này tạo ra sức mạnh cộng hưởng (Synergy) với chiến lược giữ KV Cache ở định dạng BF16 (như đề xuất ở Phần 4). FlashInfer tối ưu hóa rất mạnh cho các kiểu dữ liệu bản địa, qua đó khai thác tuyệt đối kiến trúc GQA của LFM2.5-1.2B trên phần cứng H200.⁵²

6. Các Kỹ thuật Phản tác dụng cần Tuyệt đối Tránh

Trong không gian Tối ưu của Đề bài, ban tổ chức có gợi ý một số hướng tiếp cận. Tuy nhiên, việc áp dụng mù quáng mà không xét đến ranh giới phần cứng sẽ dẫn đến thảm họa ERS. Dưới đây là các kỹ thuật phải đưa vào danh sách đen (Blacklist).

6.1. Giải mã Đầu cơ (Speculative Decoding)

Speculative Decoding là một kỹ thuật nổi bật cho phép một hệ thống (hoặc mô hình nháp) dự đoán (guess) trước một chuỗi các token tiếp theo, sau đó để mô hình chính kiểm chứng (verify) lại toàn bộ chuỗi đó trong một nhịp duy nhất. Công nghệ này có các biến thể không cần mô hình nháp (model-free) như N-gram Speculation, Prompt Lookup Decoding, hoặc Suffix Decoding.⁵⁶

Nhiều người sẽ lầm tưởng rằng N-gram Speculation hoặc Prompt Lookup là "chìa khóa vàng" vì chúng giúp tăng mạnh số token sinh ra mỗi giây. Tuy nhiên, **tuyệt đối không được kích hoạt Speculative Decoding cho hệ thống này.**

- **Lý luận Vi kiến trúc:** Các biến thể Model-free như Prompt Lookup hoặc Suffix Decoding hoạt động dựa trên cơ chế tìm kiếm chuỗi mẫu (pattern matching) trực tiếp trên CPU và bộ nhớ hệ thống (System RAM).⁵⁶ CPU sẽ phải liên tục quét và đối sánh các n-gram từ prompt và đề xuất token.
- **Thực tiễn Hệ thống:** Việc xử lý đối sánh chuỗi n-gram cho 70 ngữ cảnh đầu vào có kích thước lên đến 4000 token đòi hỏi năng lực xử lý đơn luồng (Single-thread performance) của CPU rất lớn. Khi hệ thống bị khóa cứng tại 3 lõi CPU và đã phải vật lộn duy trì I/O mạng cho lượng truy cập Poisson, việc đẩy thêm tác vụ Speculation lên CPU sẽ dẫn đến sự sụp đổ hiệu năng (Performance Collapse). CPU sẽ cạn kiệt toàn bộ tài nguyên, các tiến trình kiểm chứng (verification) sẽ bị nghẽn mạng, khiến TPOT tăng vọt một cách thảm hại.⁵⁶ Hãy để GPU H200 tự xử lý công việc của nó, đừng nhờ CPU "đoán mò".

6.2. Hoán đổi Bộ nhớ xuống RAM / NVMe (CPU Offloading)

Gợi ý về việc "Offloading xuống CPU/NVMe" chỉ có tác dụng khi VRAM không đủ chứa KV Cache.¹³ Hệ thống máy chủ có 8GB RAM, trong khi VRAM của MiG H200 là 18GB. NVMe và RAM hệ thống kết nối với GPU qua bus PCIe (ví dụ: PCIe Gen 4/5), có băng thông giới hạn (khoảng 32 - 64 GB/s) và độ trễ cao hơn gấp hàng trăm lần so với bộ nhớ HBM3e tích hợp trên bo mạch H200 (băng thông ~4.8 TB/s).¹³ Việc đẩy dữ liệu từ không gian 4.8 TB/s xuống không gian 64 GB/s là tự sát về mặt độ trễ.

- **Chiến lược Can thiệp:** Phải vô hiệu hóa hoàn toàn cơ chế swap bằng cờ --swap-space 0 để ép vLLM giữ mọi thứ khít trên VRAM, tránh mọi rủi ro về độ trễ sao chép qua PCIe.

7. Cấu hình Cực đại hóa VRAM và Lập lịch Hệ thống Sản xuất

Hệ thống chấm điểm sẽ cho một lượng truy cập đến theo phân phối Poisson với 70 hội thoại đồng thời. Sự kiện "bùng nổ" (burst) này đòi hỏi hệ thống phải có khả năng hấp thụ toàn bộ khối lượng công việc ngay lập tức mà không xếp hàng chờ đợi (Queueing). Việc xếp hàng sẽ phá hủy hoàn toàn điểm số TTFT.

7.1. Cấu hình Batching Tiên phong (Aggressive Batching)

Thay vì tối ưu (tuning) theo hướng dè dặt, hệ thống cần được cấu hình mở rộng không gian Batching bằng các tham số quyết liệt nhất, ép GPU H200 phải hấp thụ tối đa tài nguyên có thể trong một nhịp Prefill:

- `--max-num-seqs 128` (hoặc 256): Số lượng chuỗi (sequences) hoạt động song song tối đa phải được thiết lập vượt xa đỉnh (peak) của luồng Poisson (70 hội thoại). Việc mở rộng kích thước PagedAttention này tránh tình trạng các request bị đưa vào trạng thái bị hoãn (Preempted) do cơ chế quản lý bộ nhớ, đảm bảo ERS tuyệt đối cho tham số TTFT. LFM2.5-1.2B đủ nhỏ để cho phép con số này lớn hơn nữa trên 18GB VRAM.¹³
- `--max-num-batched-tokens 32768`: Do VRAM trống tới hơn 12GB, việc đặt giới hạn xử lý số lượng token trong 1 bước duy nhất lên con số cực lớn giúp hệ thống mỗi (prefill) toàn bộ 70 yêu cầu đầu vào cùng một lúc. Giả sử 70 yêu cầu x 400 token = 28.000 token. Một ngưỡng lớn hơn 32K đảm bảo CPU chỉ phát ra duy nhất 1 lệnh CUDA kernel cho toàn bộ lô dữ liệu thay vì băm nhỏ chúng ra.¹³ Hạn mức này cũng tuân thủ cấu trúc CUDA graphs khi chunked prefill bị tắt.

7.2. Ứng dụng Semantic Caching và Prefix Caching

Bài toán có 15 hội thoại primer (mỗi ngữ cảnh/warm-up) chạy trước khi bắt đầu tính điểm. Các lượt hội thoại multi-turn thường chia sẻ một khối lượng lớn các token ở phần mở đầu (như system prompt hoặc lịch sử hội thoại trước đó). Cờ `--enable-prefix-caching` là yếu tố then chốt.¹³ Khi GPU VRAM còn trống quá nhiều, cơ chế này thực tế hoạt động như một "vùng đệm lưu trữ không giới hạn". Cấu trúc cây Radix Tree của vLLM sẽ nhận diện các khối dữ liệu trùng lặp và liên kết thẳng các yêu cầu mới vào KV Cache đã có sẵn.¹³ Khi kết hợp với cấu trúc cực kỳ gọn nhẹ của KV Cache LFM2.5 (chỉ 6 lớp GQA), Prefix Caching biến giai đoạn Prefill của các luồng hội thoại phía sau thành một thao tác tra cứu bộ nhớ (memory lookup) với thời gian TTFT gần như bằng 0.¹

8. Tổng hợp Giải pháp Kiến trúc và Kịch bản Triển khai (Comprehensive Execution Plan)

Để tích hợp toàn bộ các nghiên cứu và phân tích trên vào cấu hình triển khai cuối cùng nhằm đẩy ERS chạm ngưỡng tối đa (chấp nhận hy sinh Accuracy 100% trong vòng Online), hệ thống được thiết kế theo các hướng dẫn sau đây. Đây là một bộ giải pháp phá vỡ các tư duy chuẩn mực thông thường, được "đo ni đóng giày" cho nghịch lý "Phần cứng H200 MiG bị ngẽn ở CPU 3 cores":

Bảng 2: Ma trận Thiết kế Cấu trúc Tối ưu hóa

Lĩnh vực	Giải pháp Mặc định / Phổ biến	Đề xuất Kiến trúc Cốt lõi	Giải thích Cơ sở Vi kiến trúc
----------	-------------------------------	---------------------------	-------------------------------

Giao tiếp Frontend	TCP Socket (Localhost qua Uvicorn)	Unix Domain Socket (UDS)	Giao tiếp thẳng qua bộ nhớ kernel (/tmp/vllm.sock), loại bỏ TCP/IP Network Stack, cứu rỗi CPU. ²⁷
Cấp phát RAM Hệ thống	glibc malloc mặc định	TCMalloc (LD_PRELOAD)	Sử dụng Thread-Caching Malloc tránh phân mảnh 8GB RAM, loại bỏ hiện tượng khóa luồng trong tác vụ đa tiến trình Python/Rust. ³⁹
Định dạng KV Cache	Ép kiểu FP8 (--kv-cache-dtype fp8)	BF16 / Auto (auto)	VRAM H200 đang dư thừa >12GB. Việc ép kiểu xuống FP8 tạo overhead tính toán (Casting & Scaling) vô nghĩa, làm chậm TPOT. ⁴⁸
Cơ chế Prefill	Chunked Prefill (--enable-chunked-prefill)	Tắt Chunked Prefill	Kích thước tensor động (dynamic shapes) của chunked prefill vô hiệu hóa CUDA Graphs, khiến CPU liên tục phải lập lịch kernel. ¹⁴
Đồ thị Điện toán GPU	Bị kìm hãm / Eager Mode	Bắt buộc CUDA Graphs (--enforce-eager=False)	Giúp launch hàng loạt hạt nhân toán học lên GPU H200 bằng 1 lệnh duy nhất từ CPU, vượt qua rào cản 3 Cores CPU. ⁷
Khung Thực thi	V1 Model Runner	V2 Model Runner	Kích hoạt

(Engine)	(Phụ thuộc CPU)	(MRV2)	VLLM_USE_V2_MODEL_RUNNER=1. Đẩy toàn bộ tác vụ tính toán sổ sách, batching, metadata từ CPU xuống thực thi trực tiếp bằng Triton kernels trên VRAM GPU. ²⁰
Đồng bộ CPU-GPU	Bước-theo-bước (Step-by-step Sync)	Lập lịch Đa bước (--num-scheduler-steps 10)	Giảm 90% chi phí liên lạc giữa GPU và CPU qua bus PCIe. Cổ máy GPU tự sinh 10 token liên tục trước khi phải cập nhật với bộ điều phối CPU. ¹³
Lỗi Attention (Backend)	FlashAttention-3 / XFormers	FlashInfer	Tối ưu chuyên biệt cho cấu trúc Block-sparse PagedAttention và các cơ chế chia sẻ GQA đặc thù của kiến trúc LFM2.5. ²

Cấu trúc Khởi chạy Hệ thống Sản xuất (Production Launch Config)

Sự kết hợp của các cấu hình sau đây cần được tích hợp vào file docker-compose.yml hoặc bash script thực thi trong Image khởi chạy (Entrypoint). Hệ thống này tuyệt đối tuân thủ các quy định chống gian lận (Anti-Cheating): không can thiệp trọng số, không gọi mạng bên ngoài, và serving một cách trung thực.

Phần 1: Cấu hình Biến Môi trường Hệ thống (Environment Variables)

Bash

1. Kích hoạt Rust Frontend và thay thế bộ phận Tokenizer để tăng cường I/O

export VLLM_USE_RUST_FRONTEND=

```
export TOKIO_WORKER_THREADS=
export VLLM_USE_FASTTOKENS=
```

2. Hạn chế tối đa sự tranh chấp luồng OS (Thread thrashing) của OpenMP

```
export OMP_NUM_THREADS=
export MKL_NUM_THREADS=
export OPENBLAS_NUM_THREADS=
```

3. Nâng cấp lên kiến trúc lõi MRV2 để giải phóng 100% CPU khỏi bookkeeping

```
export VLLM_USE_V2_MODEL_RUNNER=
```

4. Sử dụng FlashInfer làm lõi xử lý Attention chuyên biệt cho cấu trúc GQA

```
export VLLM_ATTENTION_BACKEND=FLASHINFER
```

5. Cấu hình cấp phát bộ nhớ RAM hiệu năng cao để sống sót trên 8GB RAM

(Giả định libtcmalloc-minimal4 đã được cài đặt trong Dockerfile)

```
export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libtcmalloc_minimal.so.4 $LD_PRELOAD
```

Phần 2: Lệnh Khởi chạy Máy chủ Inference (CLI Engine Arguments)

Thay vì các cấu hình truyền thống, lệnh khởi chạy được ép xung bằng các chỉ số Batching cực đại và loại bỏ toàn bộ các bước an toàn/đồng bộ gây chậm trễ:

```
Bash
vllm serve LiquidAI/LM2.5-1.2B-Instruct
--uds /tmp/vllm.sock
--tokenizer-mode hf
--quantization fp8
--kv-cache-dtype auto
--max-model-len 4096
--max-num-batched-tokens 65536
--max-num-seqs 256
--enable-prefix-caching
--enforce-eager False
--disable-async-output-proc
--num-scheduler-steps 10
--swap-space 0
--disable-log-stats
--disable-log-request
```

Chú giải Cấu hình Nâng cao:

- `--quantization fp8`: Vẫn giữ lượng tử hóa cho mô hình (Model Weights) để tối đa hóa băng thông bộ nhớ của H200 (Tensor Cores W8A8).
- `--kv-cache-dtype auto`: Từ chối lượng tử hóa KV Cache xuống FP8 để tránh quá tải phép toán (Compute overhead) cho GPU.
- `--disable-async-output-proc`: Tắt quá trình xử lý đầu ra bất đồng bộ, dồn toàn bộ sức mạnh cho tuyến chính.
- `--disable-log-stats / --disable-log-requests`: Loại bỏ triệt để các tác vụ I/O ghi log (File System I/O) không cần thiết để bảo vệ chu kỳ CPU.³¹
- `--num-scheduler-steps 10`: Ép GPU tự duy trì nhịp độ sinh mã liên tục (Multi-step DBO) mà không cần hỏi ý kiến CPU sau mỗi token sinh ra, kéo TPOT sập sàn (tiệm cận ngưỡng 1ms).¹³
- `--uds /tmp/vllm.sock`: Sử dụng Unix Domain Socket truyền dữ liệu thô thẳng qua Kernel.
- `--enforce-eager False`: Ép hệ thống capture tĩnh toàn bộ CUDA Graphs, hấp thụ 70 luồng Poisson qua băng thông `--max-num-batched-tokens 65536`.

Việc tái cấu trúc này tạo ra một hệ thống serving dị biệt: nó từ bỏ tham vọng bảo tồn độ chính

xác học thuật (Accuracy Gate sẽ phạt điểm Δ sau này) nhưng đổi lại là một cỗ máy nghiền nát luồng phân phối Poisson với chỉ số TTFT (tiệm cận giới hạn vật lý 10ms của Floor) và TPOT (áp sát ngưỡng 1ms) tốt nhất có thể, khai thác tận cùng giới hạn của phân vùng MiG H200 trong khi vẫn "sống sót" hoàn hảo trên tài nguyên CPU cục đoạn.

Nguồn trích dẫn

1. Deploy Liquid AI LFM2 Models (LFM2-8B-A1B, LFM2-2.6B) on GPU Cloud: Hybrid Architecture Guide (2026) | Spheron Blog, truy cập vào tháng 7 17, 2026, <https://www.spheron.network/blog/liquid-foundation-models-lfm-deployment-gpu-cloud-2026/>
2. Deploy FlashInfer on GPU Cloud: LLM Inference Kernels for vLLM and SGLang (2026 Guide) | Spheron Blog, truy cập vào tháng 7 17, 2026, <https://www.spheron.network/blog/deploy-flashinfer-gpu-cloud-llm-inference-kernels/>
3. hadad/LFM2.5-1.2B - Ollama, truy cập vào tháng 7 17, 2026, <https://ollama.com/hadad/LFM2.5-1.2B>
4. LFM2 Technical Report - arXiv, truy cập vào tháng 7 17, 2026, <https://arxiv.org/html/2511.23404v1>
5. Characterizing CPU-Induced Slowdowns in Multi-GPU LLM Inference - arXiv, truy cập vào tháng 7 17, 2026, <https://arxiv.org/html/2603.22774v1>
6. Improving FLOPS utilization with smaller models · Issue #1375 · vllm-project/vllm - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/1375>
7. Accelerating PyTorch with CUDA Graphs, truy cập vào tháng 7 17, 2026, <https://pytorch.org/blog/accelerating-pytorch-with-cuda-graphs/>

8. Liquid AI's LFM2 Just Dropped — Here's How to Run It on Vast.ai, truy cập vào tháng 7 17, 2026, <https://vast.ai/article/liquid-ai-lfm2-run-on-vast-ai>
9. LiquidAI/LFM2.5-1.2B-Instruct - Hugging Face, truy cập vào tháng 7 17, 2026, <https://huggingface.co/LiquidAI/LFM2.5-1.2B-Instruct>
10. lfm2 - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/stable/api/vllm/model_executor/models/lfm2/
11. lfm2 - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/v0.25.1/api/vllm/model_executor/models/lfm2/
12. Supported Models - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/v0.19.1/models/supported_models/
13. vLLM Throughput Optimization-1: Basic of vLLM Parameters | by Kaige - Medium, truy cập vào tháng 7 17, 2026, <https://medium.com/@kaige.yang0110/vllm-throughput-optimization-1-basic-of-vllm-parameters-c39ace00a519>
14. Cudagraph in V1 - V1 Feedback - vLLM Forums, truy cập vào tháng 7 17, 2026, <https://discuss.vllm.ai/t/cudagraph-in-v1/1016>
15. Features - vLLM Documentation, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/stable/features/>
16. vLLM V1 Alpha: A major upgrade to vLLM's core architecture - Red Hat Developer, truy cập vào tháng 7 17, 2026, <https://developers.redhat.com/articles/2025/01/28/vllm-v1-a-major-upgrade-vllm-s-core-architecture>
17. vllm.config.vllm, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/stable/api/vllm/config/vllm/>
18. torch.compile and CUDA Graphs for LLM Inference: Production PyTorch 2.6 Guide (2026), truy cập vào tháng 7 17, 2026, <https://www.spheron.network/blog/torch-compile-cuda-graphs-llm-inference-pytorch-2-6/>
19. [Bug]: TurboQuant KV + spec-decode + chunked-prefill crashes CUDA graph capture at query_start_loc.tolist() in continuation-prefill path (Qwen3-Next hybrid dense) · Issue #40807 · vllm-project/vllm - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/40807>
20. Model Runner V2 Design Document - vLLM, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/latest/design/model_runner_v2/
21. Model Runner V2: A Modular and Faster Core for vLLM, truy cập vào tháng 7 17, 2026, <https://vllm.ai/blog/2026-03-24-mrv2>
22. Dual Batch Overlap - vLLM, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/v0.11.2/design/dbo/>
23. [Roadmap] Rust Frontend Feature Parity · Issue #44280 · vllm-project/vllm - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/44280>
24. [RFC]: Rust front-end · Issue #40846 · vllm-project/vllm - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/40846>
25. Environment Variables - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/stable/configuration/env_vars/

26. GitHub - vllm-project/vllm-bench: High-performance Rust benchmark client for vLLM serving endpoints., truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm-bench>
27. datadog-agent/CHANGELOG.rst at main - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/DataDog/datadog-agent/blob/main/CHANGELOG.rst>
28. vllm launch render, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/latest/cli/launch/render/>
29. Vulnerability Summary for the Week of June 8, 2026 - CISA, truy cập vào tháng 7 17, 2026, <https://www.cisa.gov/news-events/bulletins/sb26-166>
30. Artificial intelligence - Lib.rs, truy cập vào tháng 7 17, 2026, <https://lib.rs/ai>
31. vllm serve - vLLM Documentation, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/v0.10.1/cli/serve.html>
32. Red Hat AI Inference Server 3.2 vLLM server arguments - Red Hat Documentation, truy cập vào tháng 7 17, 2026, https://docs.redhat.com/en/documentation/red_hat_ai_inference_server/3.2/pdf/vllm_server_arguments/Red_Hat_AI_Inference_Server-3.2-vLLM_server_arguments-en-US.pdf
33. Red Hat AI Inference 3.5 vLLM server arguments, truy cập vào tháng 7 17, 2026, https://docs.redhat.com/en/documentation/red_hat_ai_inference/3.5/pdf/vllm_server_arguments/Red_Hat_AI_Inference-3.5-vLLM_server_arguments-en-US.pdf
34. Optimization and Tuning - vLLM Documentation, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/stable/configuration/optimization/>
35. fasttokens - vLLM Documentation, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/latest/api/vllm/tokenizers/fasttokens/>
36. vllm.config.model, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/stable/api/vllm/config/model/>
37. [Bug]: VLLM_USE_FASTOKENS=1 raises ImportError - fasttokens, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/47855>
38. Engine Arguments - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/stable/configuration/engine_args/
39. [Feature]: add some pre-built CPU-only wheels for basic testing · Issue #23096 - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/23096>
40. Installation with CPU - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/v0.6.6/getting_started/cpu-installation.html
41. Optimization and Tuning - vLLM Ascend, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/projects/ascend/en/latest/developer_guide/performance_and_debug/optimization_and_tuning.html
42. Installation with CPU - vLLM! - Read the Docs, truy cập vào tháng 7 17, 2026, https://nm-vllm.readthedocs.io/en/latest/getting_started/cpu-installation.html
43. CPU - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/stable/getting_started/installation/cpu/
44. FP4 Quantization on Blackwell GPUs: Throughput, Cost, and When It's Worth It - Spheron, truy cập vào tháng 7 17, 2026, <https://www.spheron.network/blog/fp4-quantization-blackwell-gpu-cost/>

45. FP8 W8A8 - vLLM Documentation, truy cập vào tháng 7 17, 2026,
https://docs.vllm.ai/en/latest/features/quantization/llm_compressor/fp8/
46. LLM Inference Optimization and Quantization 2026 | Zynos Research, truy cập vào tháng 7 17, 2026, <https://zynos.ai/research/2026-01-15-llm-inference-optimization/>
47. How to Deploy an LLM On-Premise (2026): GPU Sizing & vLLM - Internal Technologies, truy cập vào tháng 7 17, 2026,
<https://internal.ai/how-to-deploy-llm-on-premise>
48. The Impact of Quantization on vLLM Inference Performance - GPUStack, truy cập vào tháng 7 17, 2026,
<https://docs.gpustack.ai/2.0/performance-lab/references/the-impact-of-quantization-on-vllm-inference-performance/>
49. vLLM Optimization Techniques: 5 Practical Methods to Improve Performance - Jarvis Labs, truy cập vào tháng 7 17, 2026,
<https://jarvislabs.ai/blog/vllm-optimization-techniques>
50. The State of FP8 KV-Cache and Attention Quantization in vLLM, truy cập vào tháng 7 17, 2026, <https://vllm.ai/blog/2026-04-22-fp8-kvcache>
51. vLLM for beginners: Key Features & Performance Optimization(PartII) - Cloudthrill, truy cập vào tháng 7 17, 2026,
<https://cloudthrill.ca/what-is-vllm-features>
52. flashinfer - vLLM, truy cập vào tháng 7 17, 2026,
<https://docs.vllm.ai/en/v0.11.0/api/vllm/v1/attention/backends/flashinfer.html>
53. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention - arXiv, truy cập vào tháng 7 17, 2026,
<https://arxiv.org/html/2405.04437v2>
54. vllm.config.model, truy cập vào tháng 7 17, 2026,
<https://docs.vllm.ai/en/v0.11.0/api/vllm/config/model.html>
55. Environment Variables - vLLM Documentation, truy cập vào tháng 7 17, 2026,
https://docs.vllm.ai/en/v0.11.0/configuration/env_vars.html
56. CMU CSD PhD Blog - SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications, truy cập vào tháng 7 17, 2026,
<https://www.cs.cmu.edu/~csd-phd-blog/2025/suffix-decoding/>
57. Speculative Decoding - vLLM Documentation, truy cập vào tháng 7 17, 2026,
https://docs.vllm.ai/en/stable/examples/features/speculative_decoding/
58. Fastest Speculative Decoding in vLLM with Arctic Inference and Arctic Training - Snowflake, truy cập vào tháng 7 17, 2026,
<https://www.snowflake.com/en/blog/engineering/fast-speculative-decoding-vllm-arctic/>
59. Real-Time LLMs: Optimizing Latency in Streaming - Latitude.so, truy cập vào tháng 7 17, 2026,
<https://latitude.so/blog/real-time-llms-optimizing-latency-streaming>
60. SPORK: Self-Speculative Forking to Accelerate Agentic LLM Inference - arXiv, truy cập vào tháng 7 17, 2026, <https://arxiv.org/html/2607.03333v1>
61. Inside vLLM's New KV Offloading Connector: Smarter Memory Transfer for Maximizing Inference Throughput, truy cập vào tháng 7 17, 2026,
<https://vllm.ai/blog/2026-01-08-kv-offloading-connector>

62. A100 80GB can't serve 10 concurrent users - what am I doing wrong? - Reddit, truy cập vào tháng 7 17, 2026,
https://www.reddit.com/r/LocalLLaMA/comments/1lgkhdh/a100_80gb_cant_serve_10_concurrent_users_what_am/
63. Daily Papers - Hugging Face, truy cập vào tháng 7 17, 2026,
<https://huggingface.co/papers?q=prefill-decode>